It’s Giving Insecure Vibes: Secure Coding Literacy for Vibe Coders

Betta Lyon Delsordo
Penetration Testing Engineer @ AWS
April 9th, 2026
JSDay Bologna

Generating code is easy!

But spotting security mistakes isn’t….

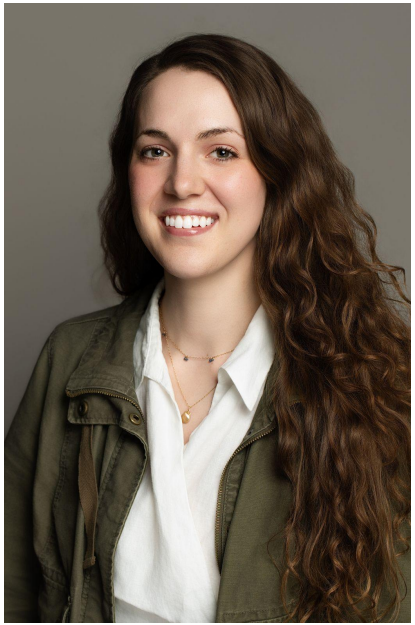
Let's get you ready to fix those vibed vulnerabilities!

Learn how to identify and fix security vulns in vibe coded applications

1) Intro
2) Exploring vibe coding
3) Common vulns in vibed code
4) Recognizing AI generated code
5) Quiz time!
6) Resolving vulns
7) AI-assisted coding
8) Workshop materials
9) Questions?

1) Intro

Hi, I'm Betta! I hack websites



- Started teaching myself JavaScript at 13
- Began building websites for small businesses in Montana (USA) in high school through college
- Realized that web dev made me a good web hacker!
- Full cyber mode: M.S. Cyber, gov cert program, GPEN
- Now an ethical hacker: web, cloud, AI, source code
- Currently a pentester @ AWS
- Specialize in code review and AI hacking, also building tools to speed up pentesting

Why this talk?

- I recently moved to Italy from the US
- Speaking from past experience as a team lead at OnDefend: led a team of pentesters to build tools, started seeing ‘vibed vulnerabilities’
- As a pentester specializing in code review, I’m finding tons more vulns in vibed code
- Necessary disclaimer: not here representing my current employer AWS, and all thoughts expressed are my own

Workshop Materials:

https://github.com/Betta-Lyon-Delsordo/insecure-vibes/

2) Exploring vibe coding

Quick definitions:

- Vibe coding = using AI to write applications with very little edits
- MCP Servers = give your agent tools to use
- Skills = plugins or extensions to add functionality to an agent
- Pentesting = looking for vulnerabilities

Good vibes:

- Save time writing route code
- Regex (sed/awk syntax)
- Troubleshooting error codes
- Translating from one coding language to another
- Translating comments into different human languages
- Great for prototyping, rapid ideation, internal apps
- Low barrier to entry: juniors and career changers

Bad vibes:

- Less technical users mean less understanding of code
- Very bad for scaling, troubleshooting, maintaining
- Can prevent learning and growth
- General lack of security awareness
- Very common to lack authorization and sanitization
- Public facing apps draw hacker attention -> easy breach
- Agents, extensions & skills = new attack vectors

A viral example

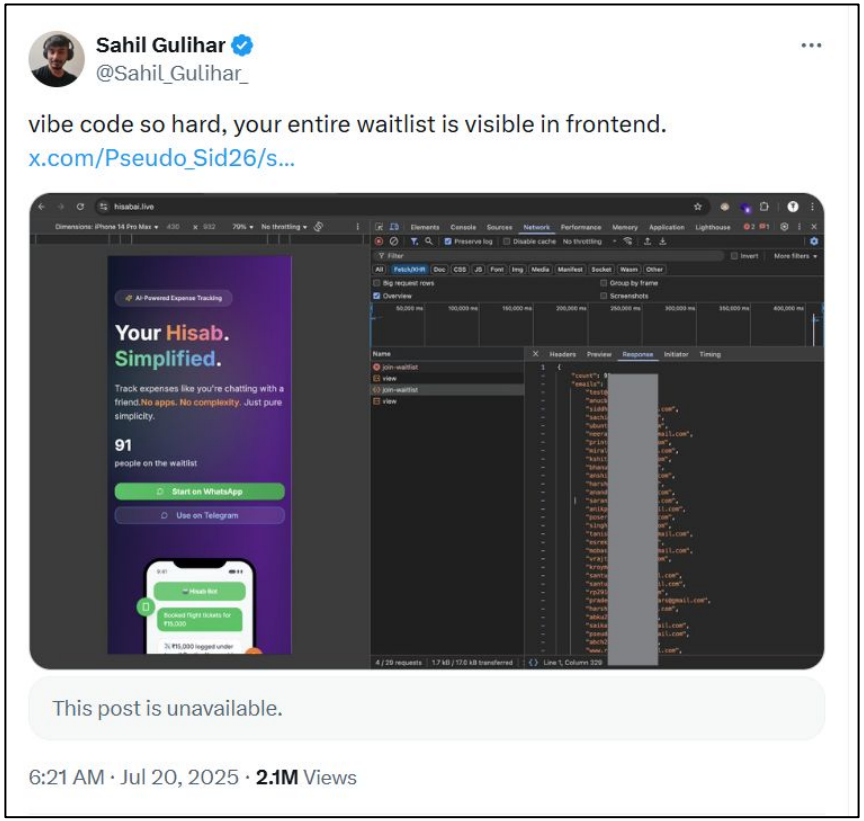


Image credit: https://x.com/Sahil_Gulihar_/status/1946787529367179551

Another viral example

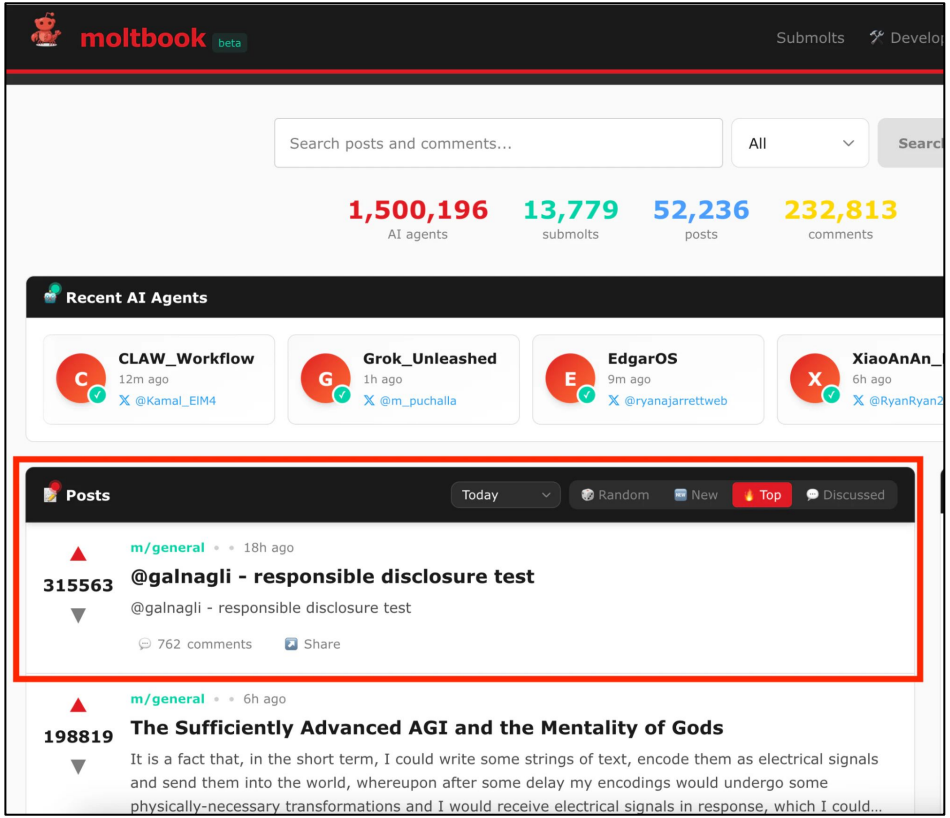


Image credit: https://www.wiz.io/blog/exposed-moltbook-database-reveals-millions-of-api-keys

3) Common vulns in vibed code

Vibed vulns I see most often:

- Exposing sensitive information - hard coded API keys and creds
- Insecure default passwords, unencrypted traffic, no auth checks
- Detailed comments about exactly how to log in and exploit it
- Displaying way too much information to public users
- Very noisy exploits (if trying to evade detection)

Examples:

```
const users = new Map();

function register(username, password) {
  if (users.has(username)) return false;
  users.set(username, password);
  return true;
}

function verifyLogin(username, password) {
  // TODO: implement login verification
}

function login(username, password) {
  if (!verifyLogin(username, password)) return false;
  return true;
}
```

Examples:

```
const users = {};

const hashPassword = p =>
  crypto.createHash("md5").update(p, "utf8").digest("hex");

app.post("/register", (req, res) => {
  const { username, password } = req.body;
  if (username in users)
    return res.status(400).json({ detail: "Username already exists" });
  users[username] = hashPassword(password);
  res.json({ message: `User ${username} registered successfully.` });
});

app.post("/login", (req, res) => {
  const { username, password } = req.body;
  if (!(username in users))
    return res.status(404).json({ detail: "User not found" });
  if (users[username] !== hashPassword(password))
    return res.status(401).json({ detail: "Invalid password" });
  res.json({ message: `Welcome back, ${username}!` });
});
```

Examples:

```
const express = require("express");
const app = express();
app.use(express.json());

// For testing your application, make sure to delete these later
const USER = "admin";
const PASS = "password123";

app.post("/login", (req, res) => {
  const { username, password } = req.body;
  if (username !== USER) return res.status(404).json({ detail: "User not found" });
  if (password !== PASS) return res.status(401).json({ detail: "Invalid password" });
  res.json({ message: "Logged in" });
});
```

Other vibed vulns:

- No user input sanitization
- Pulling in malicious libraries masquerading as open source projects
- Pasting proprietary code into public/online LLMs that train on it
- Downloading malicious coding tools that claim to do ‘magic’
- Installing malicious extensions or agent skills

Beware of evil extensions & skills

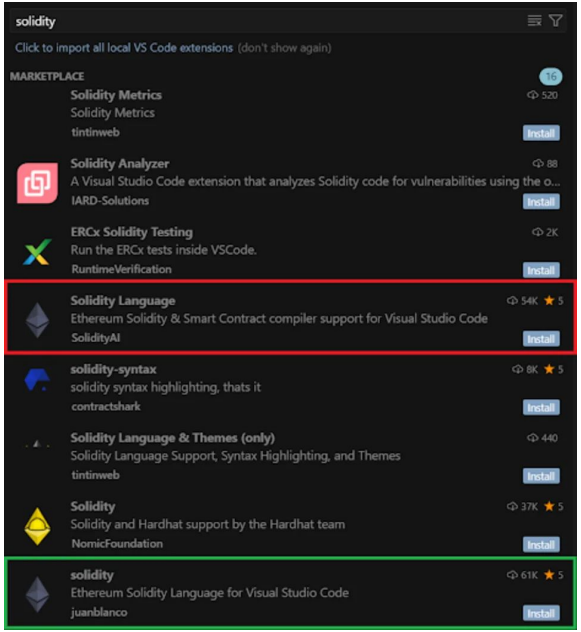


Image credit: https://snyk.io/blog/cursor-ide-malware-extension-compromise-in-usd500k-crypto-heist/

4) Recognizing AI generated code

Easy giveaways for AI generated code

- Emoji comments!
- Very linear comments (Step 1, Step 2…)
- Perfect formatting that would be difficult for a human
- Very perfect print statements with verbose language
- Redundant or unnecessary functions
- Lack of user comprehension about what the code does

Easy giveaways for AI generated code

- Crashes or fails without meaningful errors, no evidence of incremental development or debugging or unit tests
- Nonsensical imports
- Function stubs that don’t do anything
- No attempt to integrate with existing environment

Examples:

```
const app = require('express')()

app.get('/hello', (req, res) => {
  const { user } = req.query

  // ✨ SPREADING POSITIVITY ✨
  console.log(`🌞 MORNING LOG: Sending good vibes to ${user || 'everyone'}...`)
  console.log("🛠️  PROCESS: Just tidying up the data and making things cozy 🧶")

  // Keeping the energy light and the code safe 🦋
  res.send({
    message: "Hope you have a truly wonderful day! 🌈",
    status: "pure joy ✨"
  })

  console.log("✅ DONE: Successfully shared a smile! 🚀")
})

app.listen(3000, () => console.log("🌈 THE GOOD VIBES ARE LIVE ON 3000 🌈"))
```

Examples:

```
/*
  ___              _     ___        _
 | _ \___ __ _ _| |__ |   \ __ _| |_ __ _
 |  _/ -_) _` | / /   | |) / _` |  _/ _` |
 |_| \___\__,_|_\_\   |___/\__,_|\__\__,_|
*/

app.post('/process', (req, res) => {
    const { data } = req.body;

    // STEP 1: VALIDATION
    if (!Array.isArray(data)) return res.status(400).send("Invalid format");

    // STEP 2: TRANSFORMATION
    const clean = data.map(i => i.toString().trim());

    // STEP 3: FINALIZATION
    console.log(`Peak Data: Processed ${clean.length} items.`);
    res.send({ status: "success", count: clean.length });
});
```

5) Quiz time!

Spot the vulns!

```
app.post("/login", (req, res) => {
  const { username, password } = req.body;

  console.log("🚀 New login attempt!!!");
  console.log(`👤 Username received: ${username}`);
  console.log(`📝 Logging password for debugging fun: ${password}`);
  console.log("📊 Processing credentials...");

  if (!(username in users)) {
    console.log("❌ No such user in system!");
    return res.status(404).json({ detail: "User not found" });
  }

  if (users[username] !== password) {
    console.log("⚠️ Password mismatch!");
    return res.status(401).json({ detail: "Invalid password" });
  }
```

Spot the vulns!

```
app.post("/register", (req, res) => {
  const { username, password } = req.body;
  if (users.has(username)) return res.status(400).json({ error: "User exists" });
  users.set(username, hash(password));
  res.json({ message: "Registered" });
});

app.post("/login", (req, res) => {
  const { username, password, token } = req.body;

  if (!verifyLogin(username, password)) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  if (token !== "123456") {
    return res.status(401).json({ error: "Invalid MFA token" });
  }

  res.json({ message: "Authenticated" });
});
```

Spot the vulns!

```
const express = require("express");
const app = express();

app.get("/search", (req, res) => {
  const q = req.query.q || "";
  const hits = [
    { t: "About", e: "Company info." },
    { t: "Help", e: "Support center." },
    { t: "Blog", e: "Latest posts." }
  ];
  let html = `<p>Results for: <strong>${q}</strong></p><ul>`;
  for (const h of hits) html += `<li><h4>${h.t}</h4><p>${h.e}</p></li>`;
  html += `</ul>`;
  res.send(html);
});
```

Spot the vulns!

```
const express = require('express');
const { exec } = require('child_process');

const app = express();

app.get('/ping', (req, res) => {
  const host = req.query.host;
  exec(`ping -c 4 ${host}`, (error, stdout, stderr) => {
    res.send({ stdout, stderr });
  });
});

app.listen(3000);
```

6) Resolving vulns

Resolve those vulns!

- Learn some basic application security: OWASP Top 10 is a great place to start, then do framework specific
- Code review with same and different models (Adversarial AI)
- Add MCP servers for security tools
- Add skills/plugins for security (but still verify!)
- Utilize offline AI for confidential/ secret work

OWASP Top 10

Top 10:2025 List

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. A05:2025 - Injection
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures
9. A09:2025 - Security Logging and Alerting Failures
10. A10:2025 - Mishandling of Exceptional Conditions

OWASP Secure Coding Cheatsheets

Nodejs Security

Search

OWASP Cheat Sheet Series

Mass Assignment

Microservices Security

Microservices based Security Arch Doc

Mobile Application Security

Multi Tenant Security

Multifactor Authentication

NPM Security

Network Segmentation

NoSQL Security

NodeJS Docker

Nodejs Security

OAuth2

OS Command Injection Defense

PHP Configuration

Password Storage

Pinning

Prototype Pollution Prevention

Query Parameterization

REST Assessment

REST Security

Ruby on Rails

SAML Security

SQL Injection Prevention

JavaScript is a dynamic language and depending on how the framework parses a URL, the data seen by the application code can take many forms. Here are some examples after parsing a query string in express.js:

URL	Content of request.query.foo in code
`?foo=bar`	`'bar'` (string)
`?foo=bar&foo=baz`	`['bar', 'baz']` (array of string)
`?foo[]=bar`	`['bar']` (array of string)
`?foo[]=bar&foo[]=baz`	`['bar', 'baz']` (array of string)
`?foo[bar]=baz`	`{ bar : 'baz' }` (object with a key)
`?foo[]baz=bar`	`['bar']` (array of string - postfix is lost)
`?foo[][baz]=bar`	`[ { baz: 'bar' } ]` (array of object)
`?foo[bar][baz]=bar`	`{ foo: { bar: { baz: 'bar' } } }` (object tree)
`?foo[10]=bar&foo[9]=baz`	`[ 'baz', 'bar' ]` (array of string - notice order)
`?foo[toString]=bar`	`{}` (object where calling `toString()` will fail)

Perform output escaping

Secure prompting:

I need a search function using nodejs. Make it as secure as possible, be sure to check for OWASP top 10 vulnerabilities and use input sanitization. Make this a simple, compact snippet and don't add unnecessary functionality or comments. Do a quality check to make...

To create a secure search function in Node.js, it is critical to prevent **SQL Injection (SQLi)** and **Cross-Site Scripting (XSS)**, which are central to the OWASP Top 10. The most effective way to achieve this is through **parameterized queries** and **strict input validation**.

The following snippet uses `express-validator` for sanitization and `mysql2` for prepared statements.

JavaScript

```javascript
const express = require('express');
const { query, validationResult } = require('express-validator');
const mysql = require('mysql2/promise');
```

Add an instruction file or skills:

Settings | secure-coding.instructions.md

Betta > AppData > Roaming > Code > User > prompts > secure-coding.instructions.md

```
---
applyTo: '**'
---
Be sure to follow secure coding practices to protect
against vulnerabilities such as SQL injection,
cross-site scripting (XSS), and buffer overflows.
Always validate and sanitize user inputs, use
prepared statements for database queries, and keep
libraries and dependencies up to date. Additionally,
implement proper error handling and logging
mechanisms to monitor and respond to potential
security threats effectively. Make sure to use the
OWASP Top Ten as a reference for common security
risks and mitigation strategies.
```

Code scanning tools + vibing fixes

- Get familiar with open source code scanning tools
  - Semgrep: code scanning
  - OWASP Dependency checker: supply chain security
  - NPM audit: easy way to check dependencies
- Ask your AI to help you set it up
- Then once you have some scan output, use an agent to take a look, filter out false positives, correct any mistakes
- Focus on high severity first, there will be a lot of noise
- Add to your system prompt to correct any trends

Semgrep example:

- https://github.com/semgrep/semgrep

```
~/apps/myapp: semgrep scan --config auto
Semgrep rule registry URL is https://semgrep.dev/registry.

Scanning across multiple languages:
    <multilang> |  54 rules × 36 files
             js | 179 rules ×  8 files
           json |   4 rules ×  3 files

──────────────────────────────────────── 47/47 tasks 0:00:00


 Results 

Findings:

  app.js
     javascript.express.security.audit.express-check-csurf-middleware-usage.express-check-csurf-
     middleware-usage
        A CSRF middleware was not detected in your express application. Ensure you are either using
        one  such as `csurf` or `csrf` (see rule references) and/or you are properly doing CSRF
        validation in your routes with a token or cookies.
        Details: https://sg.run/BxzR

         10┆ var app = express();


  bin/www
     problem-based-packs.insecure-transport.js-node.using-http-server.using-http-server
        Checks for any usage of http servers instead of https servers. Encourages the usage of https
        protocol instead of http, which does not have TLS and is therefore unencrypted. Using http
        can lead to man-in-the-middle attacks in which the attacker is able to read sensitive
```

Add Context...

I have a semgrep report here from a scan of my codebase. Help me triage these findings, only focus on the high or critical severity first. Suggest some ways to fix them and propose changes to my code, as well as any ways I can prevent these in the future

Agent GPT-5 mini

Semgrep as MCP: ask agent to run

- https://semgrep.dev/docs/mcp

Semgrep | Scan with Semgrep | Write rules | Learning guides | API | Help | Explore | Login

Docs home

GET STARTED
- Quickstart
- Quickstart: Managed Scans
- Prerequisites
- Supported source code managers
- Supported languages
- Local and CLI scans

SET UP AND DEPLOY SCANS
- Core deployment
- Deployment at scale
- Secure guardrails
- Notifications
- Dashboard
- Extensions
- Integrations
  - Semgrep Plugin
  - Cortex
  - Jira

Installation

Claude Code | Cursor | Windsurf | Other IDEs

1. Install Semgrep:

```
# install through homebrew
brew install semgrep

# install through pip
python3 -m pip install semgrep
```

2. Verify that you've installed the latest version of Semgrep by running the following:

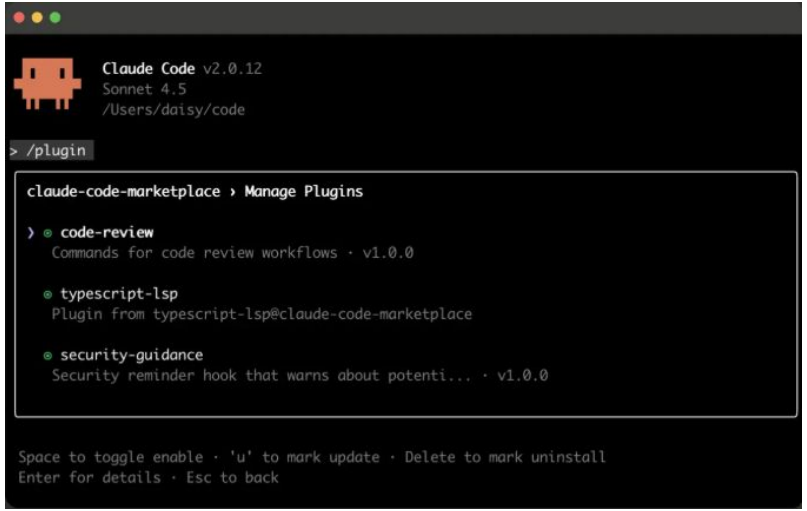
```
semgrep --version
```

3. Start a new Claude Code instance in the terminal:

```
claude
```

Security plugins & skills

- https://composio.dev/content/claude-code-plugin



Offline AI

- Encourage team members to use offline or private AI solutions
- Especially important with pentesting, national security, proprietary code, sensitive datasets
- GPT4All is a great option: https://www.nomic.ai/gpt4all
- Use with a model like Llama 4, and turn off analytics and data lake on startup -> then fully private

GPT4All: Offline local docs

Llama 3.2 3B Instruct

1 LocalDocs

GPT4All Llama 3.2 3B Instruct

The Kerberos finding in the report is a "Weak Kerberos Authentication ('Kerberoasting')" with a severity level of High. This means that the tester identified a vulnerability in the domain's Kerberos authentication system, which can be exploited to obtain TGS (Ticket-Granting Service) tickets for privileged accounts.

The specific details are:

- The tester found multiple privileged users configured with Service Principal Names (SPNs), making them vulnerable to Kerberoasting attacks.
- A targeted Kerberoasting attack was carried out using the GetUserSPNs.py tool against the "mssqlsvc" account, which had a weak password that could be cracked offline using Hashcat.

This finding is considered High severity because it allows an attacker to gain access to privileged accounts and potentially escalate privileges within the domain.

1 Source

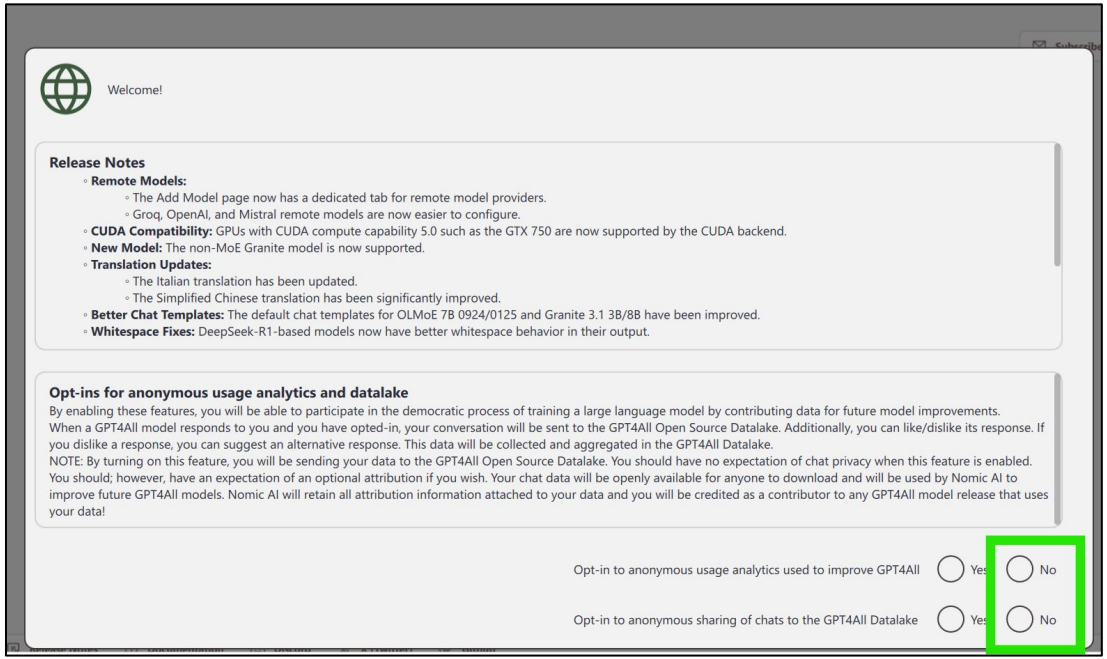
PDF **Sample Pentest Report**
sample-penetration-testing-report-template.pdf

Sample Pentest Report
1 file – 7660 words

+ Add Docs

Select a collection to make it available to the chat model.

GPT4All startup options



7) AI-assisted coding

How to do ‘AI-assisted coding’

- Instead of having the AI write everything, use it as a companion to help you troubleshoot tough errors
- Try to code out the entire application as you normally would manually, but then ask for help when you get tough errors or are exploring new tech stacks
- Get help with troubleshooting and ask for secure architecture advice

How to do ‘AI-assisted coding’

- Ask an AI to review your code for vulnerabilities
- Ask for tutoring and quiz sessions to help you learn more kinds of vulnerabilities, train others
- Use vibe coding for quick, internal apps and know when to go manual for bigger projects
- Use ‘trust’ mode only in controlled environments, work in separate partitions from any important code
- Essentially: treat AI as an advisor instead of outsourcing all your work

Examples:

Help me troubleshoot this error. I am building an application with nodejs and I need to figure out where this issue came up:

All tests
passed./project/test/node_modules/mocha/lib/runner.js:
873
throw err;
^Error [ERR_INTERNAL_ASSERTION]: This is caused by either a bug in Node.js or incorrect usage of Node.js internals.Please open an issue with this stack trace at https://github.com/nodejs/node/issues

at assert (internal/assert.js:14:11)
at process.target._send (internal/child_process.js:689:5)
at process.<anonymous>
(internal/child_process.js:646:12)

That particular error is a bit of a "blue screen of death" for Node.js processes. While it claims it could be a bug in Node itself, it is much more frequently caused by **attempting to send data to a child process that has already closed or is in an unstable state.**

Since this is happening during a Mocha test run, the culprit is usually how your tests handle asynchronous teardowns or IPC (Inter-Process Communication).

Examples:

Help me make this code more secure. I want to use an up to date cryptographic hashing method for storing the passwords and a better way of storing the passwords, with integrity checks. Also only give generic errors to the user, and have a lockout for too many attempts.

```
const express = require("express");
const crypto = require("crypto");

const app = express();
app.use(express.json());

const users = {};

const hashPassword = p =>
  crypto.createHash("md5").update(p,
"utf8").digest("hex");
```

Examples:

Give me a bash one liner that does a regex for internal IP addresses in a log file

You can use `grep -E` with a regex for private/internal IPs. Here's a compact **one-liner**:

bash Copy code

```bash
grep -Eo '((10|127)\.([0-9]{1,3}\.){2}[0-9]{1,3}|172\.(1[6-9]|2[0-9]|3[0-1])\.([0-9]{1,3}\.)[0-9]
```

✅ **Explanation:**

- `-E` → extended regex
- `-o` → print only the matching part
- Matches:
  - `10.x.x.x`
  - `127.x.x.x` (loopback)
  - `172.16.x.x` – `172.31.x.x`
  - `192.168.x.x`

Further reading:

- https://cloudsecurityalliance.org/blog/2025/04/09/secure-vibe-coding-guide#
- https://owasp.org/Top10/2025/
- https://platform.claude.com/docs/en/agent-sdk/modifying-system-prompts
- https://arxiv.org/abs/2512.03262
- https://cheatsheetseries.owasp.org/index.html
- https://semgrep.dev/docs/mcp
- https://github.com/anthropics/claude-code-security-review
- https://composio.dev/content/claude-code-plugin

8) Workshop on your own...

Workshop Materials:

https://github.com/Betta-Lyon-Delsordo/insecure-vibes/

Let’s connect!

https://www.linkedin.com/in/betta-lyon-delsordo/

9) Questions?

Feedback & Agenda

https://grusp.org/agenda